



Siete modelos de concurrency

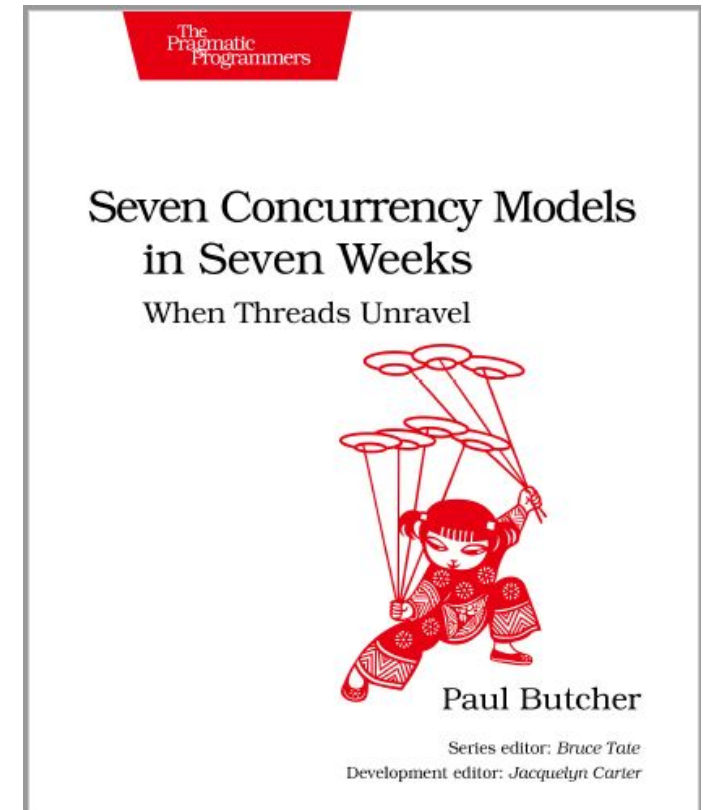
Capítulo 2 día 3

Hilos y cerrojos (Threads and locks)

Seven Concurrency Models

- *Introduction*
- *Threads and locks*
- *Functional programming*
- *The Clojure Way—separating identity and state*
- *Actors*
- *Communicating Sequential Processes*
- *Data parallelism*
- *The Lambda Architecture*

2014



Hilos y cerrojos (Threads and locks)

- La programación con hilos y cerrojos es como un vehículo Ford modelo T. Puede llevarnos del punto A al punto B, pero es primitivo, difícil de conducir, poco confiable –incluso peligroso- comparado con nuevas tecnologías.
- Olvidando un poco sus problemas, la programación con hilos y cerrojos aún se utilizan para escribir mucho software concurrente y como soporta otras tecnologías concurrentes, es bueno estudiarla.



Algo muy simple

- Los hilos y los cerrojos (Locks) son poco más que una formalización de lo que subyace el hardware realmente lo hace. Esa es su gran fuerza y, a la vez, su gran debilidad.
- Debido a que son tan simples, casi todos los lenguajes los admiten de una forma u otra, e imponen muy pocas restricciones sobre lo que se puede lograr a través de su uso.
- Pero tampoco proporcionan casi ninguna ayuda al pobre programador, haciendo que los programas sean muy difíciles de acertar en primer lugar e incluso más difícil de mantener.
- En el capítulo usan el lenguaje Java
 - Goetz, B., Peierls, T., Bloch, J., Bowbeer, J., Holmes, D., & Lea, D. (2006). *Java concurrency in practice*. Addison-Wesley Professional.
 - Urma, R.-G., Fusco, M., & Mycroft, A. (2018). *Modern Java in action: Lambdas, streams, functional and reactive programming*. Manning Publications.

Actividades propuestas para cada día

- Day 1: Mutual Exclusion and Memory Models
 - [Día 1: Exclusión mutua y modelos de memoria]
- Day 2: Beyond Intrinsic Locks
 - [Día 2: Más allá de los cerrojos intrínsecos]
- Day 3: On the Shoulders of Giants
 - [Día 3: Sobre los hombros de gigantes]



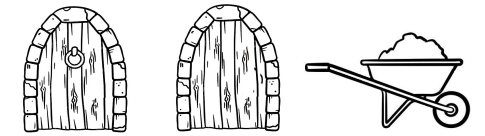
Día 3: Sobre los hombros de gigantes

- Además de los cerrojos mejorados que estudiamos en el día 2, `java.util.concurrent` contiene una colección de utilidades y estructuras de datos concurrentes de uso general, alto rendimiento y completamente depuradas. Hoy veremos que la mayoría de las veces, son una mejor opción que implementar nuestra propia solución.

Revisión de la creación de hilos

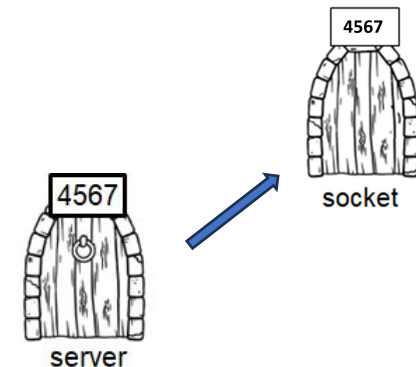
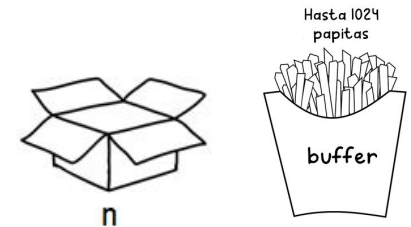
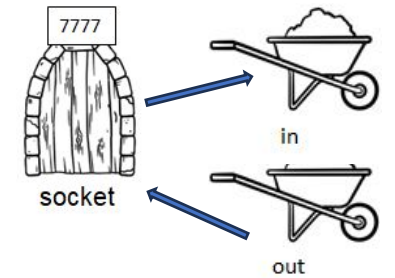
- En el día 1 vimos cómo iniciar hilos, pero resulta que rara vez tiene sentido crear hilos directamente. Aquí, por ejemplo, hay un servidor muy simple que repite todo lo que se envía:

```
public class EchoServer {  
  
    public static void main(String[] args) throws IOException {  
  
        class ConnectionHandler implements Runnable {  
            InputStream in; OutputStream out;  
            ConnectionHandler(Socket socket) throws IOException {  
                in = socket.getInputStream();  
                out = socket.getOutputStream();  
            }  
  
            public void run() {  
                try {  
                    int n;  
                    byte[] buffer = new byte[1024];  
                    while((n = in.read(buffer)) != -1) {  
                        out.write(buffer, 0, n);  
                        out.flush();  
                    }  
                } catch (IOException e) {}  
            }  
        }  
  
        ServerSocket server = new ServerSocket(4567);  
        while (true) {  
            Socket socket = server.accept();  
            Thread handler = new Thread(new ConnectionHandler(socket));  
            handler.start();  
        }  
    }  
}
```



import
java.net.*;

import
java.io.*;



Revisión de la creación de hilos

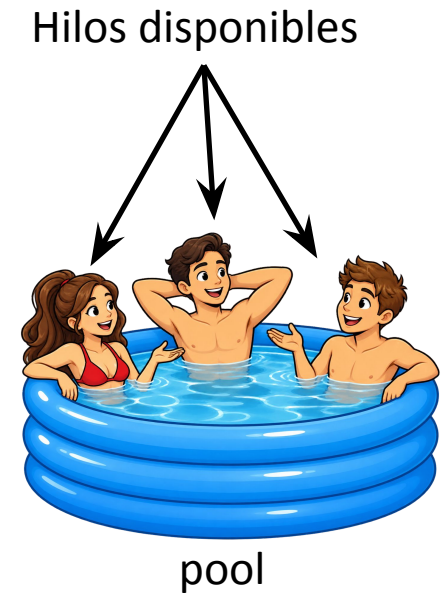
- Las líneas resaltadas siguen el patrón común de aceptar una conexión entrante y luego crear inmediatamente un nuevo hilo para manejarlo.
- Esto funciona bien, pero tiene un par de problemas.
 - Primero, aunque la creación de hilos es “barata”, no es gratis y este diseño pagará ese precio por cada conexión.
 - En segundo lugar, crea tantos hilos como conexiones; si las conexiones llegan más rápido de lo que pueden manejarse, entonces el número de hilos aumentará y el servidor se detendrá y posiblemente incluso se bloqueará.
- Esta es una oportunidad perfecta para cualquiera que quiera someter su servidor a un ataque de negación de servicios [DDoS].

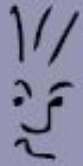
Revisión de la creación de hilos

- Podemos evitar estos problemas utilizando un pool de hilos:

```
int threadPoolSize = Runtime.getRuntime().availableProcessors() * 2;  
ExecutorService executor = Executors.newFixedThreadPool(threadPoolSize);  
while (true) {  
    Socket socket = server.accept();  
    executor.execute(new ConnectionHandler(socket));  
}
```

- Este código crea un pool de hilos con el doble de hilos que procesadores disponibles.
- Si hay más de este número de solicitudes `execute()` activas a la vez, se pondrán en cola hasta que se libere un hilo.
- Esto no solo significa que no incurrimos en la sobrecarga de la creación de hilos para cada conexión, sino que también garantiza que nuestro servidor continuará progresando frente a una carga alta (NO significa que atenderá las solicitudes entrantes lo suficientemente rápido como para mantenerse al día, sino que *algunas* de las solicitudes serán atendidas –servidas-).





Joe asks:

How Large Should My Thread Pool Be?

The optimum number of threads will vary according to the hardware you're running on, whether your threads are IO or CPU bound, what else the machine is doing at the same time, and a host of other factors.

Having said that, a good rule of thumb is that for computation-intensive tasks, you probably want to have approximately the same number of threads as available cores. Larger numbers are appropriate for IO-intensive tasks.

Beyond this rule of thumb, your best bet is to create a realistic load test and break out the stopwatch.



Copy on Write
[CoW]

Copy on Write [CoW]

- El día 1 vimos cómo llamar a los *listeners* de forma segura en un programa concurrente. Si recuerda, modificamos `updateProgress()` para hacer una copia defensiva (consulte el código en la página 19 del libro). Resulta que la biblioteca estándar de Java proporciona una solución más elegante y lista para usar en `CopyOnWriteArrayList`:

```
private CopyOnWriteArrayList<ProgressListener> listeners;
```

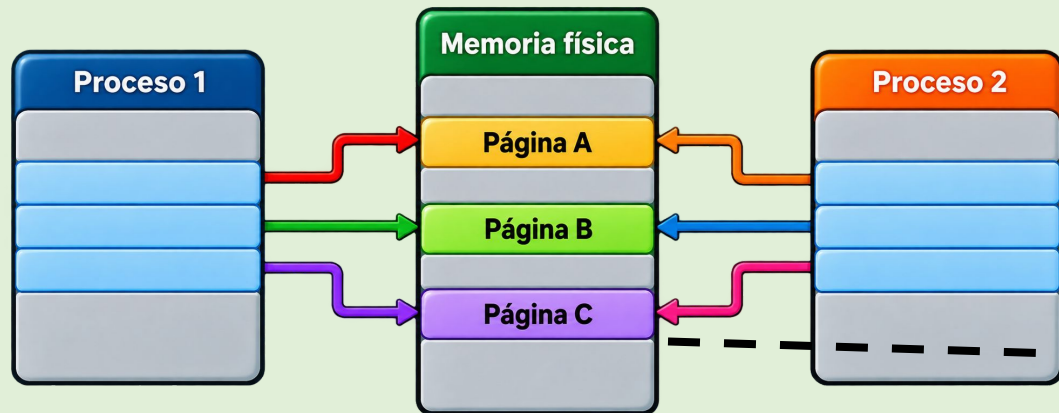
```
public void addListener(ProgressListener listener) {  
    listeners.add(listener);  
}  
public void removeListener(ProgressListener listener) {  
    listeners.remove(listener);  
}  
private void updateProgress(int n) {  
    for (ProgressListener listener: listeners)  
        listener.onProgress(n);  
}
```

Copy on Write [CoW]

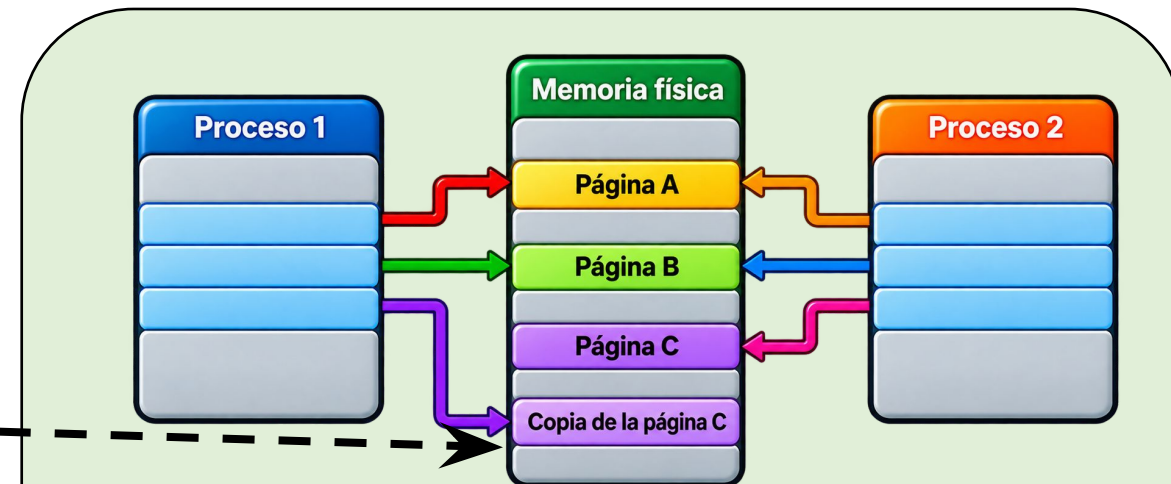
- Como sugiere su nombre, CopyOnWriteArrayList le da la vuelta a nuestra anterior estrategia de copia defensiva. En lugar de hacer una copia antes de recorrer la lista, realiza una copia cada vez que se hace un cambio. Los iteradores existentes seguirán haciendo referencia a la copia anterior. Este no es un enfoque que sea apropiado para muchos casos de uso, pero es perfecto para este.
 - Primero, como se puede ver, da como resultado un código muy claro y conciso. De hecho, aparte de la definición de *listeners*, es idéntica a la versión anterior y non-thread-safe que se presentó por primera vez en el código de la página 18 del libro.
 - En segundo lugar, es más eficiente porque no tenemos que hacer una copia cada vez que `updateProgress()` se invoca, solo cuando se modifican los *listeners* (lo que es probable que suceda con menos frecuencia).

Copy on Write [CoW] en Sistemas Operativos

- Copy on Write (COW) –en memoria virtual- permite que los procesos padre e hijo compartan inicialmente las mismas páginas en la memoria.
- Si cualquiera de los procesos modifica una página compartida, solo entonces se copia la página
 - COW permite una creación de procesos más eficiente ya que solo se copian las páginas modificadas



Estado **antes** de que el proceso 1 modifique la página c



Estado **después** de que el proceso 1 modificó la página c

Un programa
completo



Un programa completo

- Hasta ahora, hemos analizado algunas herramientas individuales de forma aislada. Ahora veremos cómo resolver un problema pequeño pero realista: ¿Cuál es la palabra más utilizada en Wikipedia?
- Debería ser bastante fácil de averiguar, simplemente descargue un volcado XML y escriba un programa para analizarlo y contar las palabras [<http://dumps.wikimedia.org/enwiki/>]. Dado que el volcad pesa alrededor de 40 GiB, llevará un tiempo; ¿Quizás podamos acelerarlo paralelizando las cosas?
- Comencemos por estimar una base inicial: ¿cuánto tiempo tarda un programa secuencial simple en contar las palabras en las primeras 100,000 páginas?

```
public class WordCount {  
    private static final HashMap<String, Integer> counts =  
        new HashMap<String, Integer>();
```

1

```
    public static void main(String[] args) throws Exception {  
        Iterable<Page> pages = new Pages(100000, "enwiki.xml");  
        for(Page page: pages) {  
            Iterable<String> words = new Words(page.getText());  
            for (String word: words)  
                countWord(word);  
        }  
    }
```

2

```
    private static void countWord(String word) {  
        Integer currentCount = counts.get(word);  
        if (currentCount == null)  
            counts.put(word, 1);  
        else  
            counts.put(word, currentCount + 1);  
    }  
}
```

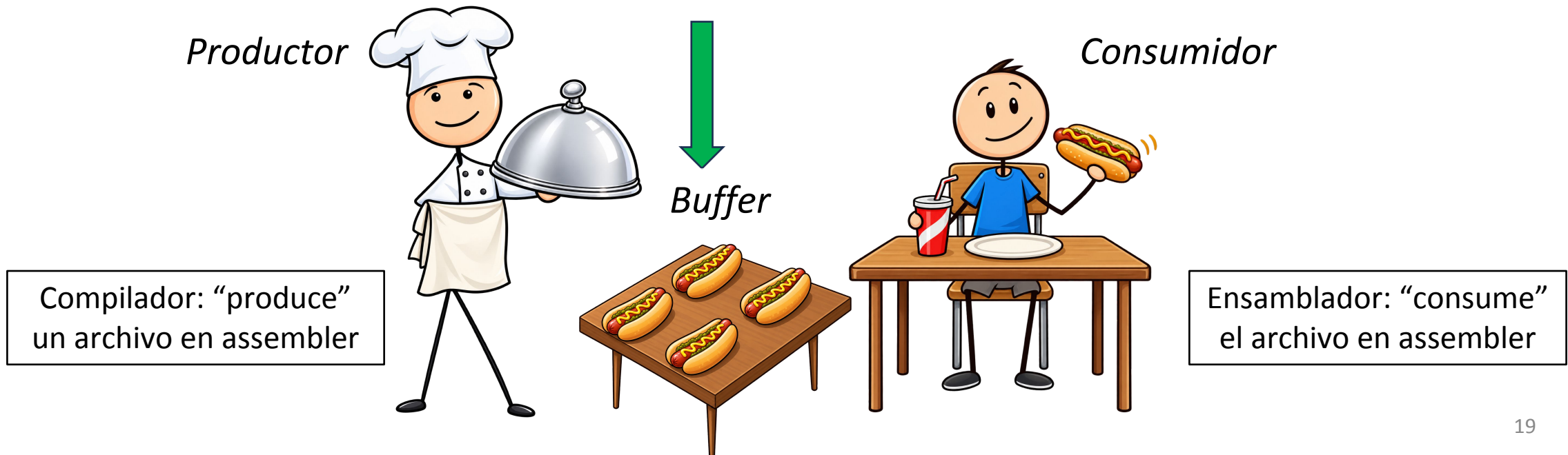
<https://guru99.es/working-with-hashmaps/>

Para usar el patrón productor-consumidor

- En mi MacBook Pro [del autor del libro], esto se ejecuta en menos de 105 segundos.
- Entonces, ¿por dónde comenzamos con una versión paralela?
- Cada iteración del bucle principal realiza dos tareas:
 1. primero analiza lo suficiente del XML para producir una página, y
 2. luego consume esa página contando las palabras en su texto.
- Existe un patrón clásico que se puede aplicar a este tipo de problema: el patrón productor-consumidor. En lugar de un solo hilo que alterna entre producir valores y luego consumirlos, creamos dos hilos, un productor y un consumidor.

Paradigma del productor-consumidor

- Paradigma para procesos que cooperan, el proceso *productor* (*producer*) produce información que es consumida por un proceso *consumidor* (*consumer*)
 - **unbounded-buffer (buffer no acotado)** NO establece límites prácticos en relación con el tamaño del buffer
 - **bounded-buffer (buffer acotado)** asume que hay un tamaño de buffer fijo

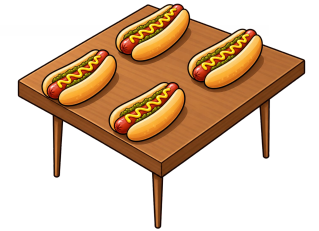


Analizador (parser) como productor

- Aquí hay un analizador implementado como productor
- El método `run()` contiene el bucle externo de nuestra solución secuencial, pero en lugar de contar las palabras en la página recién analizada, lo agrega al final de una cola.

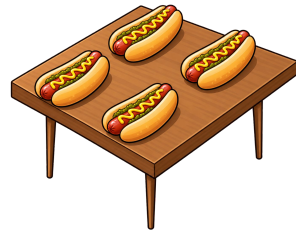


```
class Parser implements Runnable {  
    private BlockingQueue<Page> queue;  
  
    public Parser(BlockingQueue<Page> queue) {  
        this.queue = queue;  
    }  
  
    public void run() {  
        try {  
            > Iterable<Page> pages = new Pages(100000, "enwiki.xml");  
            > for (Page page: pages)  
            >     queue.put(page);  
        } catch (Exception e) { e.printStackTrace(); }  
    }  
}
```



La cola (*queue*) es el *buffer* (una mesa)

Consumidor



La cola (*queue*) es el
buffer (una mesa)

- Aquí está el consumidor correspondiente
- Como era de esperar, contiene el bucle interno de nuestra solución secuencial, tomando su entrada de la cola.



```
class Counter implements Runnable {  
    private BlockingQueue<Page> queue;  
    private Map<String, Integer> counts;  
    public Counter(BlockingQueue<Page> queue,  
                   Map<String, Integer> counts) {  
        this.queue = queue;  
        this.counts = counts;  
    }  
  
    public void run() {  
        try {  
            while(true) {  
                ➤ Page page = queue.take();  
                if (page.isPoisonPill())  
                    break;  
  
                ➤ Iterable<String> words = new Words(page.getText());  
                ➤ for (String word: words)  
                    countWord(word);  
            }  
        } catch (Exception e) { e.printStackTrace(); }  
    }  
}
```

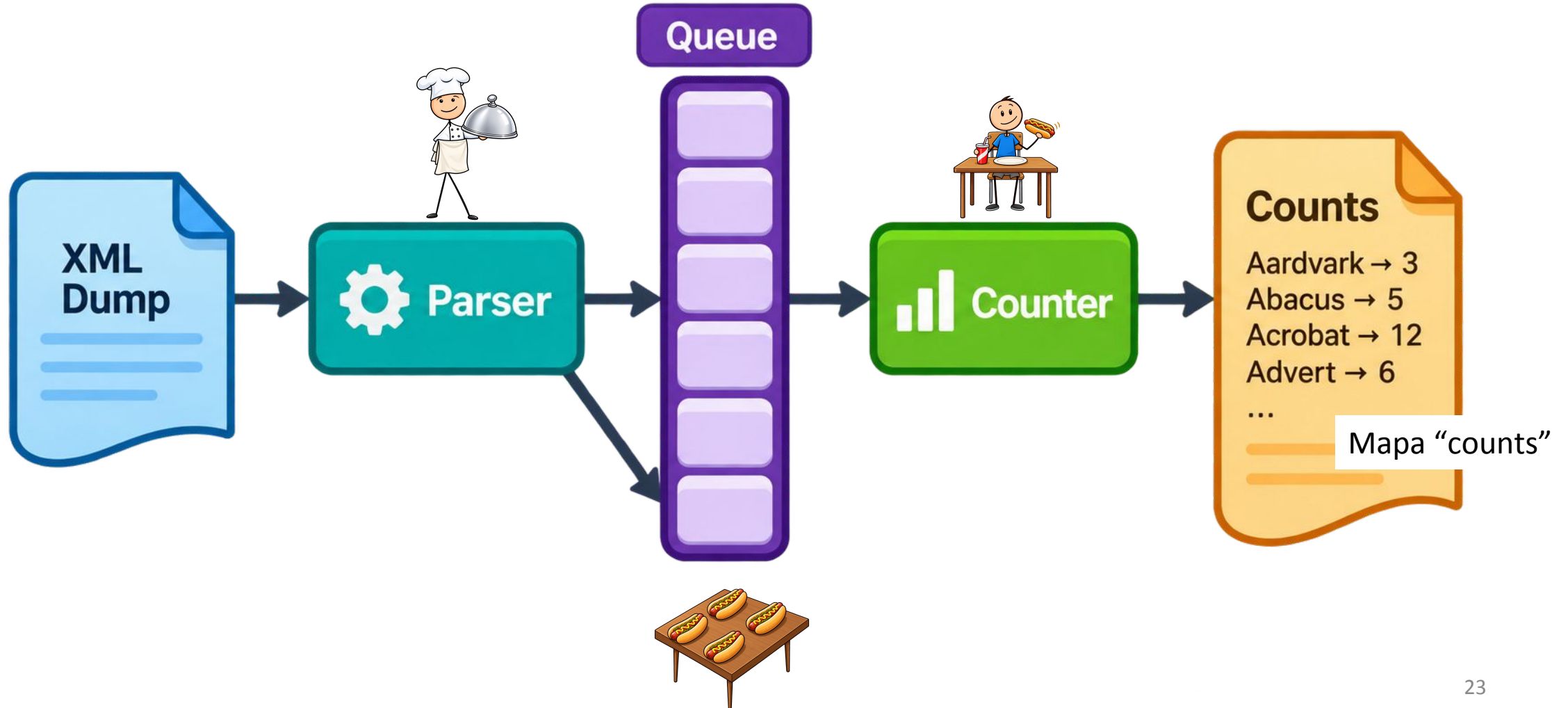
```
ArrayBlockingQueue<Page> queue = new ArrayBlockingQueue<Page>(100);  
HashMap<String, Integer> counts = new HashMap<String, Integer>();
```

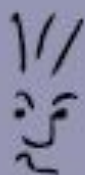
```
Thread counter = new Thread(new Counter(queue, counts));  
Thread parser = new Thread(new Parser(queue));
```

```
counter.start();  
parser.start();  
parser.join();  
queue.put(new PoisonPill());  
counter.join();
```

- Finalmente, aquí hay una versión modificada de nuestro bucle principal que crea estos dos hilos
- ArrayBlockingQueue de java.util.concurrent es una cola concurrente ideal para implementar el patrón productor-consumidor. No solo proporciona los métodos concurrentes put() y take() eficientes, sino que estos métodos se bloquearán cuando sea necesario.
- Intentar ejecutar take() para una cola vacía se bloqueará hasta que la cola NO esté vacía, e intentar put() en una cola llena se bloqueará hasta que la cola tenga espacio disponible.

Arquitectura solución productor-consumidor





Joe asks:

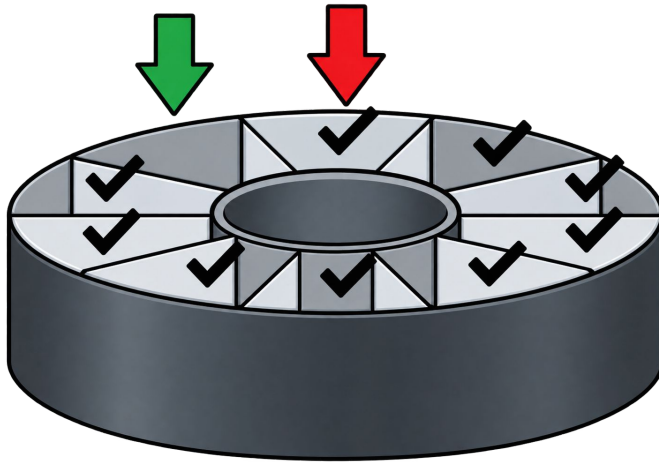
Why a Blocking Queue?

As well as blocking queues, `java.util.concurrent` provides `ConcurrentLinkedQueue`, an unbounded, wait-free, and nonblocking queue. That sounds like a very desirable set of attributes, so why isn't it a good choice for this problem?

The issue is that the producer and consumer may not (almost certainly will not) run at the same speed. In particular, if the producer runs faster than the consumer, the queue will get larger and larger. Given that the Wikipedia dump we're parsing is around 40 GiB, that could easily result in the queue becoming too large to fit in memory.

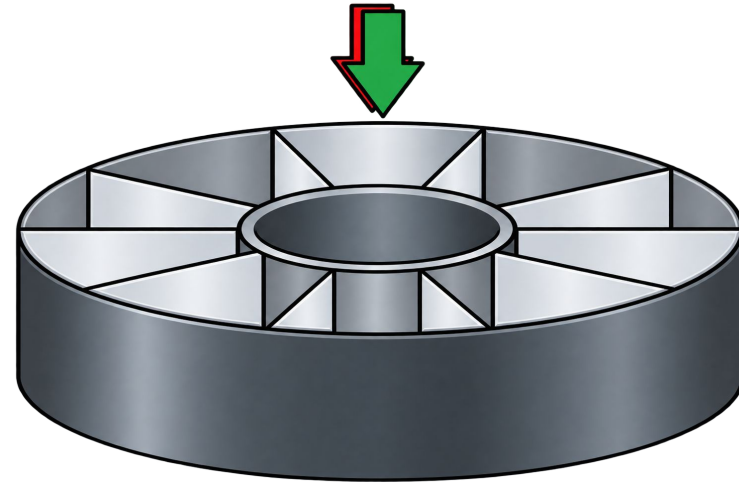
Using a blocking queue, by contrast, will allow the producer to get ahead of the consumer, but not too far.

Se debe controlar que el buffer esté lleno o que esté vacío



Buffer lleno

Debe bloquearse para el productor
(no hay espacio para colocar nuevos ítems)



Buffer vacío

Debe bloquearse para el consumidor
(no hay ítems para “consumir”)

¿Cuándo debe terminar el consumidor?

- Otro aspecto interesante de esta solución es cómo el consumidor sabe cuándo salir:

```
if (page.isPoisonPill())  
    break;
```

- Como sugiere su nombre, una PoisonPill [pastilla venenosa] es un token especial que indica que se ha llegado al final de los datos disponibles y que, por tanto, el consumidor debe salir. Es muy similar a la forma en que C/C++ utiliza un carácter nulo para indicar el final de una cadena.

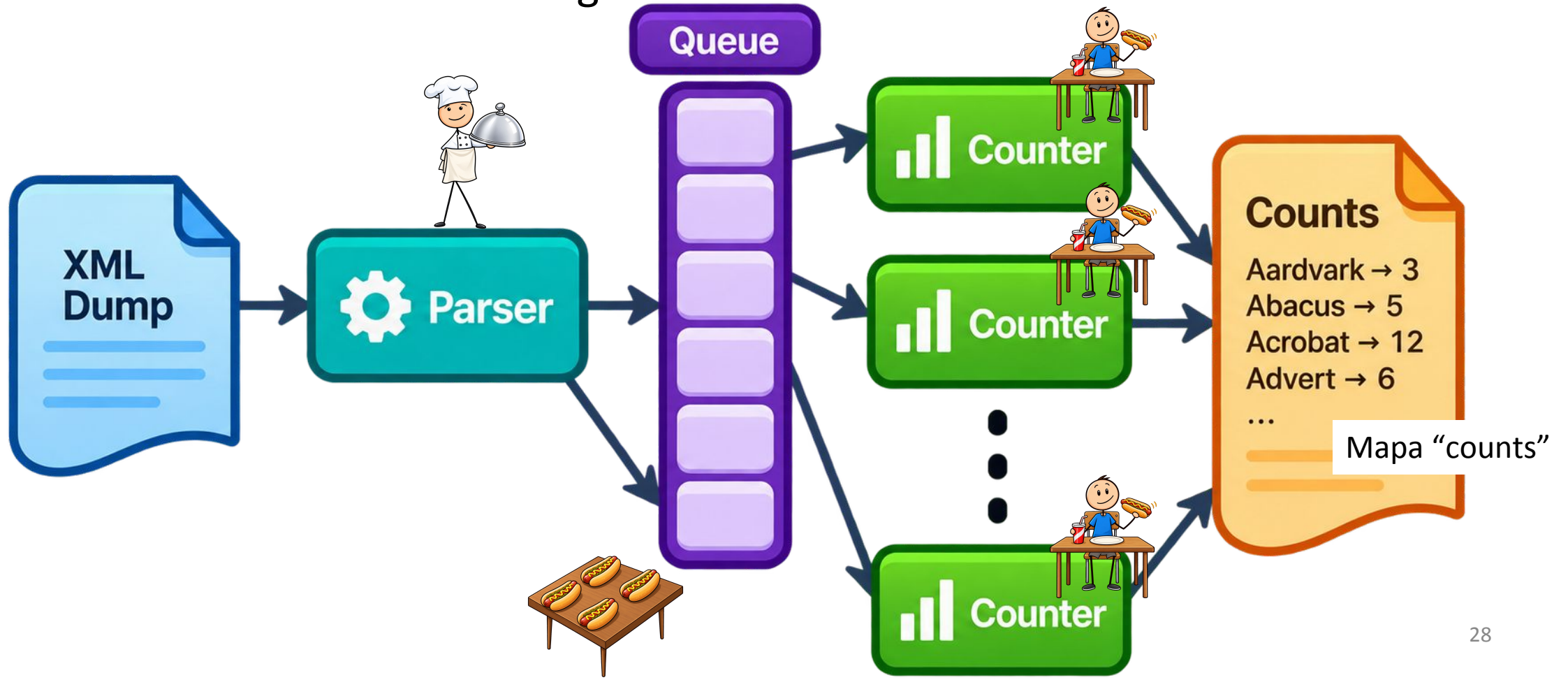


Mejorando la rapidez de las cosas

- La buena noticia es que esto ha mejorado la rapidez de las cosas; en lugar de ejecutarse en 105 segundos, esta versión se ejecuta en 95.
- Eso es genial, pero podemos hacerlo mejor. La belleza del patrón productor-consumidor es que nos permite no solo producir y consumir valores en paralelo, sino también tener múltiples productores y/o múltiples consumidores.
- Pero, ¿deberíamos concentrarnos en acelerar al productor o al consumidor?
- ¿Dónde pasa el tiempo el código? Si modifico temporalmente el código para que solo se ejecute el productor y uso mi cronómetro, encuentro que puedo analizar las primeras 100.000 páginas en unos 10 segundos.
- Esto no es sorprendente si lo piensa por un momento. La versión secuencial original del código se ejecutó en 105 segundos y la versión de productor-consumidor se ejecutó en 95. Claramente, el análisis toma 10 segundos y el conteo de palabras toma 95. Al analizar y contar en paralelo, podemos reducir el total a lo que sea es el más largo de estos, en este caso, 95 segundos.

Múltiples consumidores

- Entonces, para mejorar aún más el rendimiento, necesitamos paralelizar el proceso de conteo y tener múltiples consumidores. La siguiente figura muestra hacia dónde nos dirigimos



Sincronizar el acceso nosotros mismos

- Si vamos a tener varios hilos contando palabras simultáneamente, tendremos que encontrar una forma de sincronizar el acceso al mapa de counts.
- Lo primero que podríamos considerar es usar un *synchronized map*, retornado por el método `synchronizedMap()` en *Collections*.
Desafortunadamente, las colecciones sincronizadas no proporcionan métodos atómicos read-modify-write, por lo que esto no nos ayudará. Si queremos usar un `HashMap`, tendremos que sincronizar el acceso nosotros mismos.

countWord() modificado

- Aquí hay un método countWord() modificado que hace exactamente esto.

```
private void countWord(String word) {  
    ➤ lock.lock();  
    try {  
        Integer currentCount = counts.get(word);  
        if (currentCount == null)  
            counts.put(word, 1);  
        else  
            counts.put(word, currentCount + 1);  
    } ➤ finally { lock.unlock(); }  
}
```



Queue

Varios consumidores

```
ArrayBlockingQueue<Page> queue = new ArrayBlockingQueue<Page>(100);  
HashMap<String, Integer> counts = new HashMap<String, Integer>();  
ExecutorService executor = Executors.newCachedThreadPool();  
for (int i = 0; i < NUM_COUNTERS; ++i)  
    executor.execute(new Counter(queue, counts));  
Thread parser = new Thread(new Parser(queue));  
parser.start();  
parser.join();  
for (int i = 0; i < NUM_COUNTERS; ++i)  
    queue.put(new PoisonPill());  
executor.shutdown();  
executor.awaitTermination(10L, TimeUnit.MINUTES);
```

Aardvark → 3
Abacus → 5
Acrobat → 12
Advert → 6
...



- Y aquí hay un bucle principal modificado que ejecuta varios consumidores.
- Este es muy similar al que teníamos antes, excepto que hemos cambiado a usar un pool de hilos, que es más conveniente que administrar varios hilos nosotros mismos.
- Y debemos asegurarnos de agregar la cantidad correcta de PoisonPills [píldoras venenosas] a la cola para terminar de forma “limpia”.

¿Mejora la rapidez de ejecución?

- Todo esto se ve muy bien, pero nuestras esperanzas están a punto de desvanecerse. A continuación, se indica cuánto tiempo se tarda en ejecutar con uno y dos consumidores (la mejora en la rapidez –speedup- es relativa a la versión secuencial):

Consumers	Time (s)	Speedup
1	101	1.04
2	212	0.49



- ¿Por qué añadir otro consumidor ralentiza las cosas? ¿Y no solo un poco más lento, sino más del doble?



Mejor usar ConcurrentHashMap

Contención: “competir”
por el mismo recurso.

- La respuesta es una **contención excesiva**: demasiados hilos están intentando acceder a un único recurso compartido simultáneamente. En nuestro caso, los consumidores pasan tanto tiempo con el mapa *counts* bloqueado, que pasan más tiempo esperando a que el otro lo desbloquee de lo que pasan realmente haciendo un trabajo útil, lo que conduce a un rendimiento desastrozo.
- Felizmente, todavía no estamos vencidos. ConcurrentHashMap en `java.util.concurrent` parece exactamente lo que necesitamos.
 - No solo proporciona métodos atómicos read-modify-write, sino que ha sido diseñado para dar soporte a altos niveles de acceso simultáneo (a través de una técnica llamada *lock striping* [bloqueo de bandas]).

<https://www.geeksforgeeks.org/concurrenthashmap-in-java/>

countWord() modificado [otra vez]

- Aquí hay un countWord() modificado que usa ConcurrentHashMap

```
private void countWord(String word) {  
    while (true) {  
        Integer currentCount = counts.get(word);  
        if (currentCount == null) {  
            if (counts.putIfAbsent(word, 1) == null)  
                break;  
        } else if (counts.replace(word, currentCount, currentCount + 1)) {  
            break;  
        }  
    }  
}
```

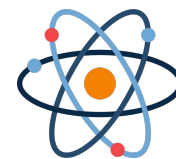
putIfAbsent() y replace()

- Vale la pena dedicar un poco de tiempo a comprender exactamente cómo funciona esto.
 - En lugar del método `put()`, ahora estamos usando una combinación de los métodos `putIfAbsent()` y `replace()`.
- Aquí está la documentación para `putIfAbsent()`:

If the specified key is not already associated with a value, associate it with the given value. This is equivalent to

```
if (!map.containsKey(key))  
    return map.put(key, value);  
else  
    return map.get(key);
```

except that the action is performed atomically.



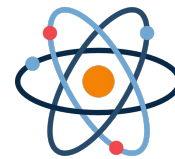
putIfAbsent() y replace()

- Y aquí está la documentación para replace()

Replaces the entry for a key only if currently mapped to a given value. This is equivalent to

```
if (map.containsKey(key) && map.get(key).equals(oldValue)) {  
    map.put(key, newValue);  
    return true;  
} else return false;
```

except that the action is performed atomically.



Mejora en la rapidez de ejecución

- Entonces, cada vez que llamamos a cualquiera de estas funciones, debemos verificar su valor de retorno para determinar si han realizado correctamente el cambio que esperábamos. Si no es así, tenemos que hacer un bucle y volver a intentarlo.
- Con esta versión, el cronómetro es mucho más amable con nosotros:

Consumers	Time (s)	Speedup
1	120	0.87
2	83	1.26
3	65	1.61
4	63	1.67
5	70	1.50
6	79	1.33



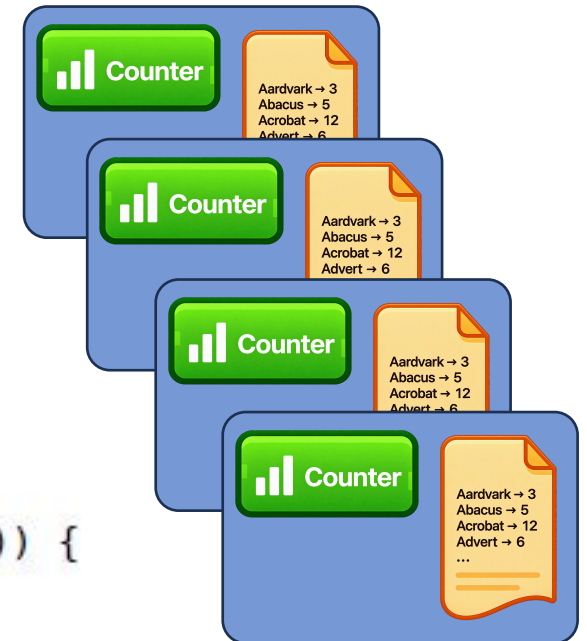
Cada consumidor con sus cuentas locales

- ¡Éxito! Esta vez, agregar más consumidores nos hace ir más rápido, al menos hasta tener más de cuatro consumidores, después de lo cual las cosas se vuelven más lentas nuevamente.
- Dicho esto, aunque 63 segundos es ciertamente más rápido que los 105 segundos que toma la versión secuencial del código, ni siquiera es el doble. Mi MacBook [del autor, en el año 2014] tiene cuatro núcleos; seguramente deberíamos poder acercarnos a una aceleración 4x.
- Con un poco de “trabajo mental”, está claro que nuestra solución está creando mucha más contención de la necesaria para el mapa counts. En lugar de que cada consumidor actualice un conjunto compartido de cuentas concurrentemente, cada uno ***debería mantener su propio conjunto local de cuentas.***

mergeCounts()

- Todo lo que tenemos que hacer es integrar [merge] estas cuentas locales antes de salir

```
private void mergeCounts() {  
    for (Map.Entry<String, Integer> e: localCounts.entrySet()) {  
        String word = e.getKey();  
        Integer count = e.getValue();  
        while (true) {  
            Integer currentCount = counts.get(word);  
            if (currentCount == null) {  
                if (counts.putIfAbsent(word, count) == null)  
                    break;  
            } else if (counts.replace(word, currentCount, currentCount + count)) {  
                break;  
            }  
        }  
    }  
}
```



Mejora en la rapidez de ejecución [otra vez]

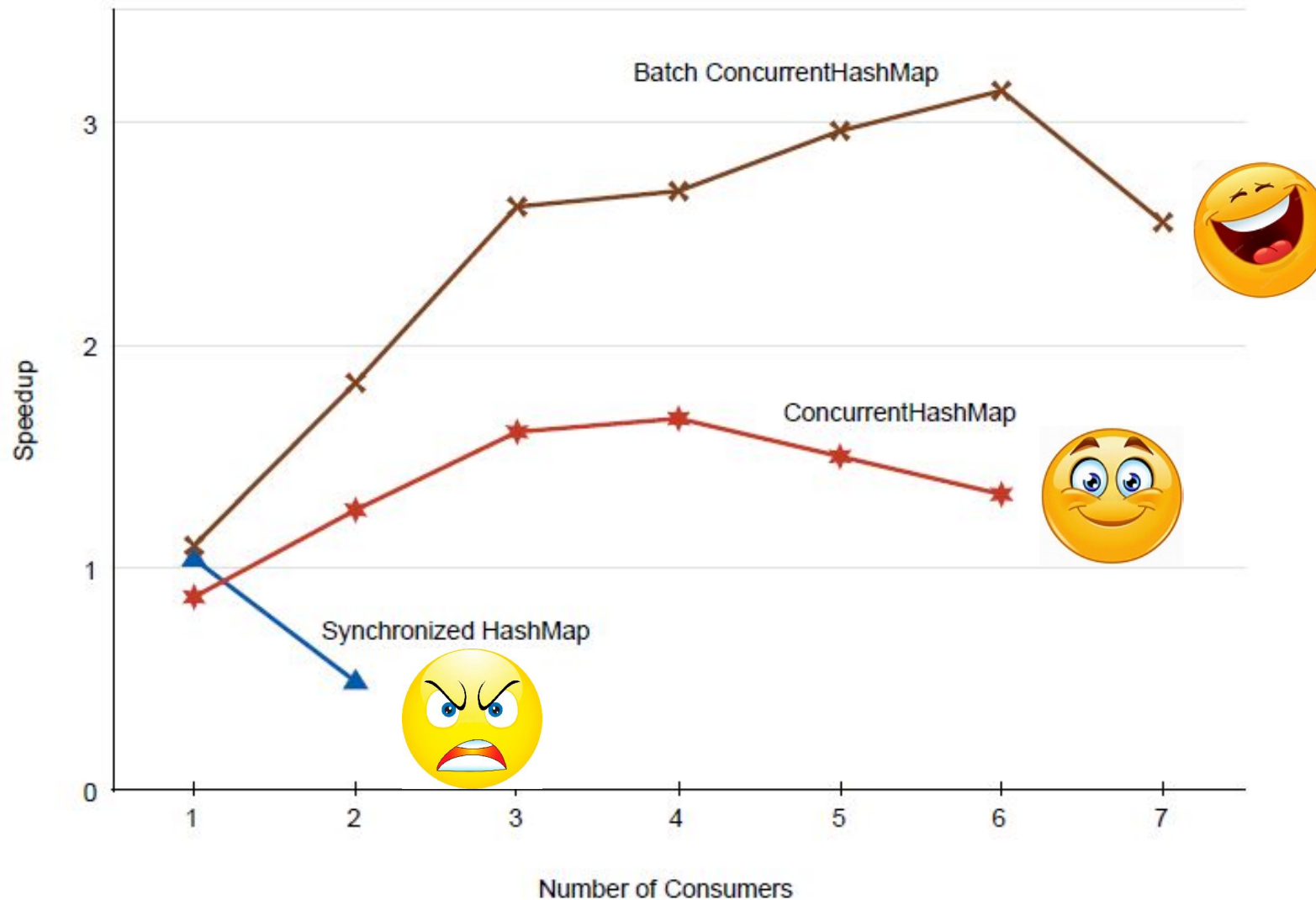
- Esto nos acerca mucho más a nuestra mejora en rapidez ideal de 4x:

Consumers	Time (s)	Speedup
1	95	1.10
2	57	1.83
3	40	2.62
4	39	2.69
5	35	2.96
6	33	3.14
7	41	2.55



- Esta versión no solo aumenta el rendimiento más rápidamente a medida que agregamos consumidores: incluso continúa mejorando en rendimiento más allá de cuatro consumidores. Esto es posible porque cada uno de los núcleos de mi MacBook [el del autor] admite dos "hyperfthreads" —`availableProcessors()` retorna el valor ocho, aunque solo hay cuatro núcleos físicos.

Rendimiento del recuento de palabras por número de consumidores



Rendimiento de las tres versiones

- Verá la curva mostrada antes una y otra vez cuando trabaje con programas paralelos.
- El rendimiento aumenta inicialmente de forma lineal y luego es seguido por un período en el que el rendimiento continúa aumentando, pero más lentamente. Con el tiempo, el rendimiento alcanzará su punto máximo y agregar más hilos solo hará que las cosas sean más lentas.
- Vale la pena tomarse un momento para reflexionar sobre lo que acabamos de hacer. Hemos creado un programa productor-consumidor relativamente sofisticado, con varios consumidores que se coordinan entre sí a través de una cola concurrente y un mapa concurrente.
- E hicimos todo esto sin ningun cerrojo explícita porque construimos sobre las facilidades proporcionadas por la biblioteca estándar.

Resumen del día 3

- Esto nos lleva al final del día 3 y al final de nuestra discusión sobre la programación con hilos y cerrojos.
- Lo que aprendimos en el día 3. Vimos cómo las facilidades proporcionadas por `java.util.concurrent` no solo facilitan la creación de programas concurrentes, sino que también hacen que esos programas sean más seguros y eficientes al hacer lo siguiente:
 - Usar pool de hilos en lugar de crear hilos directamente
 - Creación de un código de gestión de listeners más simple y eficiente con `CopyOnWriteArrayList`
 - Permitir a productores y consumidores comunicarse de manera eficiente con `ArrayBlockingQueue`
 - Soporte de acceso altamente concurrente a un mapa con `ConcurrentHashMap`

Día 3 Autoestudio

- Encontrar

- La documentación de ForkJoinPool para Java: ¿en qué se diferencia un pool fork/join de un pool de hilos [thread pool]? ¿Cuándo preferiría uno y cuándo el otro?
- ¿Qué es el robo de trabajo [work-stealing] y cuándo podría ser útil? ¿Cómo implementaría el robo de trabajo [work-stealing] con las facilidades proporcionadas por `java.util.concurrent`?
- ¿Cuál es la diferencia entre `CountDownLatch` y `CyclicBarrier`? ¿Cuándo podría usar uno y cuándo el otro?
- ¿Qué es la ley de Amdahl? ¿Qué dice acerca de la máxima aceleración teórica que podríamos obtener para nuestro algoritmo de conteo de palabras?

Día 3 Autoestudio

- Hacer

- Vuelva a escribir el código de productor-consumidor para usar una bandera de "end of data" separada en lugar de una poison pill [píldora de veneno]. Asegúrese de que su solución maneje correctamente los casos en los que el productor corre más rápido que el consumidor y viceversa. ¿Qué pasará si el consumidor ya ha intentado eliminar algo de la cola cuando se establece la marca de "end of data"? ¿Por qué cree que el enfoque de la poison pill [píldora venenosa] se usa con tanta frecuencia?
- Ejecute las diferentes versiones del programa de conteo de palabras en su computador, así como a otras versiones que pueda encontrar. ¿En qué se diferencian los gráficos de rendimiento de una computadora a otra? Si pudieras ejecutarlo en una computadora con 32 núcleos, ¿cree que vería algo cercano a una mejora en la rapidez de 32 veces?

Resumen día 3

- Cubrimos cómo crear hilos y usar los bloqueos intrínsecos integrados en cada Objeto de Java para hacer cumplir la exclusión mutua entre ellos. También vimos los tres los peligros principales de los subprocesos y bloqueos: condiciones de carrera, interbloqueo y memoria visibilidad, y discutimos algunas reglas que nos ayudan a evitarlas:
 - Sincronice todo el acceso a las variables compartidas.
 - Tanto los hilos de escritura como los de lectura necesitan usar sincronización.
 - Adquirir múltiples cerrojos en un orden global fijo.
 - No llame a métodos alienígenas mientras mantiene un candado.
 - Mantenga los cerrojos durante el menor tiempo posible.

Programas revisados

1.	Counting	Ok	
2.	CountingBetter	Ok	
3.	CountingFixed	Ok	
4.	DiningPhilosophers	Ok	
5.	DiningPhilosophersCondition	Ok	
6.	DiningPhilosophersFixed	Ok	
7.	DiningPhilosophersTimeout	Ok	
8.	EchoServer	NO	
9.	EchoServerBetter	NO	
10.	HelloWorld	Ok	
11.	HttpDownload	Ok	
12.	HttpDownloadBetter	NO	
13.	HttpDownloadFixed	Ok	
14.	Interruptible	Ok	
15.	LinkedList	Ok	
16.	Puzzle	Ok	
17.	Uninterruptible	Ok	
18.	WordCount	NO	
19.	WordCountBatchConcurrentHashMap		NO
20.	WordCountConcurrentHashMap		NO
21.	WordCountProducerConsumer		NO
22.	WordCountSynchronizedHashMap		NO

Conclusión del capítulo

- Fortalezas:
 - Se pueden aplicar a una amplia gama de problemas.
 - Pueden ser muy eficientes cuando se usan correctamente.
 - Se pueden integrar fácilmente en la mayoría de los lenguajes de programación.
- Debilidades
 - No brindan soporte directo para el paralelismo
 - Se pueden utilizar para paralelizar un algoritmo secuencial, pero introduce NO determinismo.
 - Admite solo arquitecturas de memoria compartida [share memory].
 - Si usted necesita admitir memoria distribuida (y, por extensión, distribución geográfica o resiliencia), deberá buscar en otra parte
 - La programación de hilos y cerrojo es difícil [muy poca ayuda]
 - Las pruebas [testing] y el mantenimiento son tareas difíciles

Gracias